Introduction au langage C

BTS CIEL

Sommaire

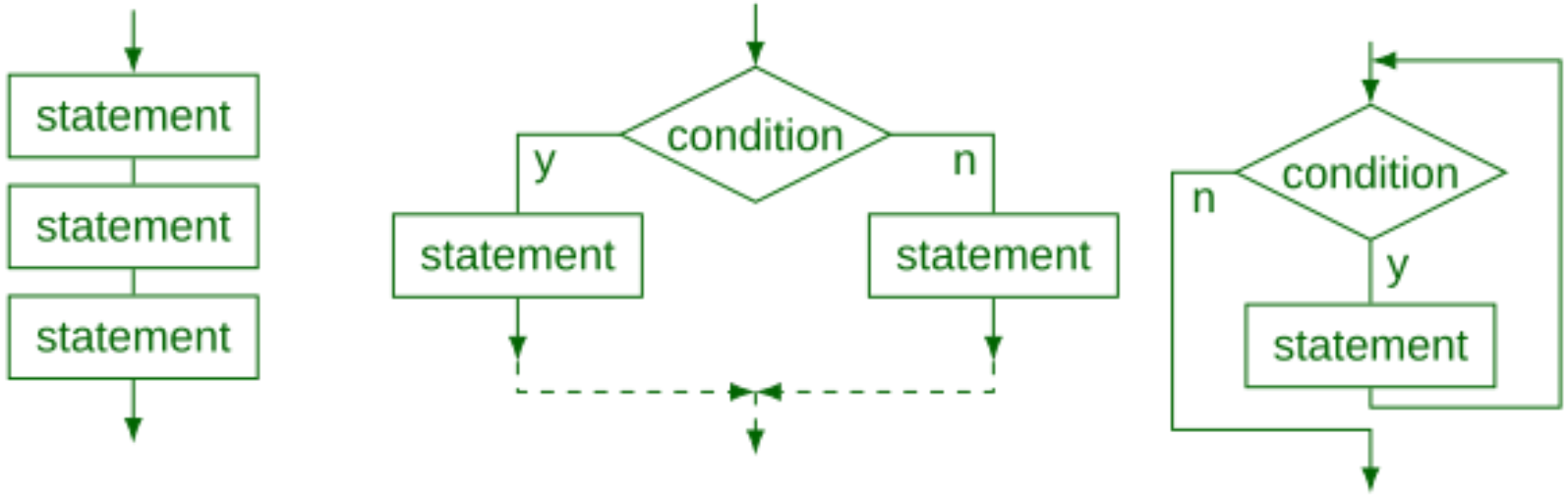
- Langage compilé
- Programmation structurée
  - Instructions, variables et blocs
  - Séquence
  - Sélection
  - Itération
- Pointeurs
- Exercices

Langage compilé

Programmation structurée

La programmation structurée définie les primitives suivantes

- ⬇️ La séquence
- 🔀 La sélection
- 🔁 L'itération



⬇️ Séquence - variable

```
int ma_variable = 10  // integer
char * mon_autre_variable = "du texte"  // char *
bool mon_bool = true  // boolean

int ma_variable; // Attention, comportement non prédictible si non initialisé
```

⬇️ Séquence - typage

Types fondamentaux :

- Numériques entiers : `char`, `short`, `int`, `long` (et variantes `signed` / `unsigned`)
- Numériques réels : `float`, `double`, `long double`
- Caractères : `char` (`char *`)
- Booléens : `_Bool` (ou `stdbool.h`)

Structures de données :

- Tableaux : `int tab[10];`
- Structures : `struct`

> ℹ️ Ces types sont directement supportés par le langage C (sans bibliothèques externes).

🔽 Séquence - typage

Python utilise un typage dit **dynamique** et **strique**.

```python
points = 3.2  # points est du type float
print("Tu as " + points + " points !")  # Génère une erreur de typage

points = int(points)  # points est maintenant du type int (entier), sa valeur est arrondie à l'unité inférieure (ici 3)
print("Tu as " + points + " points !")  # Génère une erreur de typage

print("Tu as " + str(points) + " points !")  # Plus d'erreur de typage, affiche 'Tu as 3 points !'
```

Le C utilise un typage dit **statique** et **faible**.

```c
float points = 3.2; // Typage statique : points est un float
points = (int) points; // Erreur points est de type float elle ne peut pas recevoir un int

int pts = (int) points; // Conversion explicite : float → int
printf("Tu as %s points !\n", pts); // Typage faible : conversion implicite
```

> ⚠️ En C l'erreur sera levée lors de la compilation du programme

⬇️ Séquence - typage

`Définition`

Typage Dynamique / statique
Est-ce qu'une variable de type **A** peut contenir une valeur de type **B** sans être rédéfinie.

Typage faible / fort
Est-ce qu'une variable de type **A** peut-être utilisée en tant que type **B** (sans cast explicite)

⬇️ Séquence - opérations mathématiques

En C, les types numériques de base sont :

- `int` (entier)
- `float` (réel simple précision)
- `double` (réel double précision)

Opération	Définition
`x + y`	somme de x et y
`x - y`	différence de x et y
`x * y`	produit de x et y
`x / y`	quotient de x et y
`x % y`	reste de la division (entiers uniquement)
`-x`	opposé de x

⬇️ Séquence - instruction et bloc

```c
#include <stdio.h>

int main(void) {
    int x = 10;
    {
        int y = 5;
        printf("%d\n", x + y);
    }
    return 0;
}
```

⬇️ Séquence - appel de fonction

```c
#include <stdio.h>
#include <math.h>

int main(void) {
    printf("Hello world !\n");

    int a = 10, b = 15;
    int le_max = (a > b) ? a : b;
    printf("Max = %d\n", le_max);

    double r = sqrt(16.0);
    printf("Racine carrée = %.2f\n", r);

    return 0;
}
```

🔀 Sélection - if / else if / else

```c
#include <stdio.h>

int main(void) {
    int ma_variable = 10;

    if (ma_variable == 11) {
        printf("foo\n");
    } else if (ma_variable == 12) {
        printf("or\n");
    } else {
        printf("bar\n");
    }

    return 0;
}
```

🔀 Sélection - opérations de comparaison

Opération	Définition
`<`	strictement inférieur
`<=`	inférieur ou égal
`>`	strictement supérieur
`>=`	supérieur ou égal
`==`	égal
`!=`	différent

> ⚠️ Pas d'opérateur `is` en C.
> On compare les adresses mémoire pour les pointeurs (`ptr1 == ptr2`).

🔀 Sélection - véridicité des valeurs

En C :

- `0` (zéro) est considéré comme **faux**.
- Toute autre valeur entière est **vraie**.

```
if (0) {
    printf("faux\n");
}
if (42) {
    printf("vrai\n");
}
```

🔀 Sélection - opérations booléennes

Opération	Définition
`x	
`x && y`	vrai si x et y sont vrais
`!x`	vrai si x est faux

🔀 Sélection - switch / case

```c
#include <stdio.h>

int main(void) {
    int status = 404;

    switch (status) {
        case 400:
            printf("Bad request\n");
            break;
        case 404:
            printf("Not found\n");
            break;
        case 418:
            printf("I'm a teapot\n");
            break;
        default:
            printf("Code erreur inconnu\n");
    }

    return 0;
}
```

🔁 Itération - while loop

```c
#include <stdio.h>

int main(void) {
    int i = 0;

    while (i < 10) {
        printf("%d\n", i);
        i++;
    }

    char *animals[] = {"chien", "chat", "souris"};
    i = 0;
    while (i < 3) {
        printf("%s\n", animals[i]);
        i++;
    }

    return 0;
}
```

🔁 Itération - for loop

```c
#include <stdio.h>

int main(void) {
    char *animals[] = {"chien", "chat", "souris"};

    for (int i = 0; i < 3; i++) {
        printf("%s\n", animals[i]);
    }

    return 0;
}
```

Les tableaux en C

Un **tableau** est une zone mémoire contiguë contenant plusieurs éléments du **même type**.

```c
#include <stdio.h>

int main(void) {
    int tab1[5];
    int tab2[5] = {0};
    int tab3[5] = {1, 2, 3};
    int tab4[]  = {10, 20, 30};

    printf("%d\n", tab3[0]);
    printf("%d\n", tab4[2]);

    return 0;
}
```

Les pointeurs en C

Un pointeur est une variable qui contient **l'adresse mémoire** d'une autre variable.

```c
#include <stdio.h>

int main(void) {
    int valeur = 42;
    int *ptr = &valeur;

    printf("Adresse = %p\n", ptr);
    printf("Valeur via ptr = %d\n", *ptr);

    *ptr = 99;
    printf("Nouvelle valeur = %d\n", valeur);

    return 0;
}
```

Les pointeurs en C

- déclaration d’un pointeur : `type *ptr;`
- récupérer l'adresse d'une variable `var` : `&var;`
- déréférencement (accède à la valeur pointée) : `*ptr`

Les pointeurs permettent :

- de manipuler des **tableaux** et des **chaînes**
- de gérer la **mémoire dynamique** (`malloc`, `free`)
- de passer des paramètres **par adresse** aux fonctions

Exercice 1 — chaînes et `char *`

```c
#include <stdio.h>

int main(void) {
    char nom[10];
    const char *msg = "Coucou";

    int i = 0;
    while (____) {
        nom[i] = ____;
        i++;
    }
    nom[i] = ____;

    printf("%s\n", nom);
    return 0;
}
```

Exercice 2 — opérateur ternaire

```c
#include <stdio.h>

int main(void) {
    int a = 5, b = 8;
    int max;
    if (a > b) {
        max = a;
    } else {
        max = b;
    }
    printf("Max = %d\n", max);
    return 0;
}
```

Exercice 3 — `switch` et `enum`

```c
#include <stdio.h>

enum Couleur { ROUGE, VERT, BLEU };

int main(void) {
    enum Couleur c = VERT;

    switch (c) {
        case ____:
            printf("Rouge\n");
            break;
        case ____:
            printf("Vert\n");
            break;
        case ____:
            printf("Bleu\n");
            break;
        default:
            printf("Inconnu\n");
    }
    return 0;
}
```

Exercice 4 — pointeurs et adresses

```c
#include <stdio.h>

int main(void) {
    int x = 42;
    int *ptr = ____;

    printf("Valeur de x = %d\n", ____);
    printf("Adresse de x = %p\n", ____);

    return 0;
}
```

Exercice 5 — tableau vs. pointeur

```c
#include <stdio.h>

int main(void) {
    int tab[3] = {10, 20, 30};
    int *p = ____;

    for (int i = 0; i < 3; i++) {
        printf("%d\n", ____);
        ___;
    }
    return 0;
}
```